

Python3 Performance

Course Overview & Environment

- Course created using Python 3.11 and fully compatible with Python 3.11+.
- Uses VS Code, PyCharm Professional, Pytest-benchmark, Line-profiler, Memory-profiler, and Dask.
- Focuses on Python performance optimization techniques and tools.
- Covers profiling, benchmarking, threading, async programming, and multiprocessing.
- Emphasizes practical performance analysis and optimization.

Performance Optimization Strategy

- Prevent performance issues through good architecture and efficient algorithms.
- Measure performance before making assumptions.
- Optimize only after identifying actual bottlenecks.
- Maintain comprehensive test coverage during optimization.
- Consider trade-offs such as complexity and memory usage.

```
import time
start = time.time()
for i in range(1000000):
    pass
end = time.time()
print("Execution Time:", end - start)
```

Performance Bottlenecks

- Most bottlenecks originate from CPU, Memory, Disk, or Network usage.
- CPU bottlenecks occur during heavy computations.
- Memory bottlenecks occur when applications consume excessive RAM.

- Disk bottlenecks involve slow file operations.
- Network bottlenecks appear during data transfer operations.

Performance Measurement Tools

- Windows Task Manager, Activity Monitor, and System Monitor track system resources.
- Linux provides `time` and `top` commands for monitoring.
- Python's `time` module measures execution time.
- `timeit` provides accurate benchmarking.
- `pytest-benchmark` automates performance testing.

```
import timeit
```

```
result = timeit.timeit("sum(range(1000))", number=1000)
```

```
print(result)
```

Code Profiling

- Profiling identifies exact performance bottlenecks.
- Follows the Pareto Principle where a small portion of code causes most delays.
- Eliminates guesswork during optimization.
- Helps prioritize optimization efforts.
- Provides measurable performance insights.

Event-Based Profiling

- Records every function call and event.
- Provides highly accurate performance data.
- Generates detailed execution statistics.
- Introduces significant runtime overhead.
- Suitable for deep analysis.

Statistical Profiling

- Samples program execution periodically.
- Produces approximate performance data.
- Lower overhead than event-based profiling.
- Scales better for large applications.
- Useful for production environments.

cProfile

- Recommended built-in profiler for Python.
- Lower overhead compared to profile.
- Provides function-level statistics.
- Helps identify expensive function calls.
- Generates output compatible with visualization tools.

```
import cProfile
```

```
def calculate():
```

```
    return sum(range(1000000))
```

```
cProfile.run('calculate()')
```

Line Profiler

- Measures execution time line by line.
- Uses the @profile decorator.
- Executed using kernprof.
- Identifies slow lines inside functions.
- Useful for fine-grained optimization.

```
@profile
```

```
def square_numbers():
```

```
    total = 0
```

```
    for i in range(10000):
```

```
total += i * i
```

```
return total
```

Memory Profiler

- Tracks memory usage line by line.
- Uses the @profile decorator.
- Detects memory leaks and excessive allocations.
- Works with mprof for visualization.
- Helps reduce memory consumption.

```
@profile
```

```
def create_list():
```

```
    data = [i for i in range(100000)]
```

```
    return data
```

Big O Notation

- Measures algorithm scalability independently of hardware.
- Evaluates performance as input size grows.
- $O(1)$ is the fastest complexity.
- $O(n^2)$ is significantly slower for large inputs.
- Helps choose efficient algorithms.

```
# O(1)
```

```
value = my_list[0]
```

```
# O(n)
```

```
for item in my_list:
```

```
    print(item)
```

Lists and Arrays

- Lists are dynamic and resizable.

- Accessing and appending items is $O(1)$.
- Removing or searching elements is $O(n)$.
- Arrays use less memory for specific data types.
- NumPy arrays are optimized for numerical operations.

```
numbers = [1, 2, 3]
```

```
numbers.append(4)
```

```
import numpy as np
```

```
arr = np.array([1, 2, 3])
```

Sets and Tuples

- Sets store unique immutable values.
- Membership testing in sets is $O(1)$.
- Tuples are immutable and memory efficient.
- Tuples are generally faster than lists.
- Sets are useful for duplicate removal.

```
numbers = {1, 2, 3}
```

```
print(2 in numbers)
```

```
person = ("John", 25)
```

Queues and Deques

- Queues support FIFO and LIFO operations.
- Designed for multithreaded environments.
- Deques allow efficient operations at both ends.
- Deques provide $O(1)$ append and pop operations.
- Random index access in deques is $O(n)$.

```
from collections import deque
```

```
dq = deque([1, 2, 3])
```

```
dq.appendleft(0)
```

Dictionaries

- Store data as key-value pairs.
- Keys must be unique and hashable.
- Access operations are $O(1)$.
- Faster searches than lists.
- Commonly used for lookups and mappings.

```
student = {
```

```
    "name": "Alice",
```

```
    "age": 20
```

```
}
```

```
print(student["name"])
```

Data Classes and Named Tuples

- Named tuples provide readable immutable structures.
- Data classes reduce boilerplate code.
- Data classes support methods and inheritance.
- Mutable by default with optional immutability.
- Faster development for data-centric classes.

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Student:
```

```
    name: str
```

```
    age: int
```

Caching

- Stores computed results for reuse.
- Reduces expensive computations.
- Requires additional memory.
- May return stale data if not managed.
- Useful for repeated function calls.

```
from functools import lru_cache
```

```
@lru_cache(maxsize=100)
```

```
def fibonacci(n):
```

```
    if n < 2:
```

```
        return n
```

```
    return fibonacci(n-1) + fibonacci(n-2)
```

For Loops vs List Comprehensions

- For loops are flexible and readable.
- List comprehensions are concise.
- List comprehensions are generally faster.
- Ideal for simple transformations.
- Create new lists efficiently.

```
squares = [x*x for x in range(10)]
```

Generator Expressions

- Generate values on demand.
- Consume very little memory.
- Suitable for large datasets.
- Can only be iterated once.
- Do not support random access.

```
squares = (x*x for x in range(10))
```

```
for num in squares:
```

```
    print(num)
```

String Concatenation

- '+' operator is simple but slower.
- f-strings are fast and readable.
- join() is efficient for many strings.
- Performance depends on use case.
- join() scales best for large datasets.

```
words = ["Python", "is", "fast"]
```

```
result = " ".join(words)
```

Permission vs Forgiveness

- Permission checks conditions before action.
- Forgiveness handles exceptions after failure.
- try/except avoids race condition issues.
- Exception handling is Pythonic.
- Improves robustness.

```
try:
```

```
    value = data["name"]
```

```
except KeyError:
```

```
    print("Missing key")
```

Functions and Performance

- Function calls introduce overhead.
- Self-contained functions can be faster.
- Reusability improves maintainability.

- Avoid premature optimization.
- Balance performance and readability.

Numerical Calculations

- NumPy is optimized for scientific computing.
- Pandas relies heavily on NumPy.
- Vectorized operations outperform loops.
- Ideal for statistics and machine learning.
- Reduces execution time significantly.

```
import numpy as np
```

```
arr = np.array([1, 2, 3])
```

```
print(arr * 2)
```

Alternative Python Interpreters

- CPython is the official interpreter.
- PyPy offers significant speed improvements.
- Cython converts Python into C extensions.
- Jython runs on the JVM.
- Pyston focuses on performance enhancements.

Optimization Risks

- Can reduce code readability.
- May introduce new bugs.
- Sometimes offers minimal gains.
- Can increase memory usage.
- Requires careful cost-benefit analysis.

Using Threads

- Threads enable concurrent execution.
- Share the same memory space.
- Lightweight compared to processes.
- Suitable for I/O-bound tasks.
- Vulnerable to race conditions.

```
import threading

def task():

    print("Running")

thread = threading.Thread(target=task)

thread.start()

thread.join()
```

Thread Synchronization

- Prevents concurrent data corruption.
- Locks protect shared resources.
- Semaphores limit resource access.
- Condition variables coordinate threads.
- Helps avoid race conditions.

```
import threading

lock = threading.Lock()

with lock:

    print("Protected section")
```

Global Interpreter Lock (GIL)

- Allows only one thread to execute Python bytecode.
- Limits CPU-bound multithreading performance.
- Released during I/O operations.

- Affects standard CPython.
- Encourages multiprocessing for CPU workloads.

When to Use Threads

- Network requests and downloads.
- File operations and database access.
- Waiting for external resources.
- Lightweight concurrency requirements.
- Simple asynchronous behavior.

When to Avoid Threads

- CPU-intensive calculations.
- Complex synchronization requirements.
- Applications without I/O waiting.
- High-performance parallel computing.
- Situations prone to race conditions.

Asynchronous Programming

- Uses coroutines for concurrency.
- Reduces overhead compared to threads.
- Uses `async` and `await` keywords.
- Managed by an event loop.
- Best for I/O-heavy applications.

```
import asyncio
```

```
async def hello():
```

```
    await asyncio.sleep(1)
```

```
    print("Hello")
```

```
asyncio.run(hello())
```

When to Use Asyncio

- High-volume network requests.
- Concurrent I/O operations.
- Data processing pipelines.
- Lightweight task scheduling.
- Avoiding thread synchronization.

When to Avoid Asyncio

- CPU-intensive processing.
- Blocking third-party libraries.
- Legacy synchronous frameworks.
- Small simple applications.
- Heavy dependence on blocking code.

Multiprocessing

- Uses multiple processes instead of threads.
- Bypasses the GIL completely.
- Suitable for CPU-intensive tasks.
- Each process has independent memory.
- Provides true parallelism.

```
from multiprocessing import Process
```

```
def task():
```

```
    print("Running")
```

```
p = Process(target=task)
```

```
p.start()
```

```
p.join()
```

Inter-Process Communication

- Processes do not share memory.
- Pipes allow direct communication.
- Queues safely transfer data.
- Supports producer-consumer patterns.
- Enables process synchronization.

```
from multiprocessing import Queue
```

```
queue = Queue()
```

```
queue.put("Hello")
```

```
print(queue.get())
```

Multiprocessing Best Practices

- Limit the number of processes.
- Handle exceptions carefully.
- Terminate processes cleanly.
- Monitor application performance.
- Protect shared resources.

Distributed Computing Frameworks

- Celery distributes background tasks.
- Dask scales NumPy and Pandas workloads.
- Ray supports large-scale Python applications.
- Kubernetes manages containerized deployments.
- Enables scaling across multiple machines.

```
import dask.array as da
```

```
arr = da.arange(1000000, chunks=10000)
```

```
print(arr.sum().compute())
```